UNESP - IBILCE
Departamento de Ciência da Computação e Estatísticas
Disciplina de Compiladores - Bacharelado em Ciência da Computação

Analisador Léxico - Trabalho 1

Alunos: Gabriel Martins Brum e João Carlos Gonçalves Nobre
Professora: Profa. Dra. Renata Spolon Lobato

Outubro
2026

1 Descrição da Linguagem Regular

A linguagem regular desenvolvida para este analisador léxico é baseada nos requisitos do Trabalho 1 e composta pelas seguintes definições:

- **Identificadores:** Iniciam obrigatoriamente com uma letra ou sublinhado (`[a-zA-Z_]`), podendo ser seguidos por letras, números ou sublinhados adicionais.
- **Números Inteiros:** Formados por uma sequência contínua de um ou mais dígitos (`[0-9]+`).
- **Números Reais:** Definidos por uma parte inteira de um ou mais dígitos, seguida de um ponto separador (`.`) e uma parte fracionária (`[0-9]+`).
- **Operadores:** A linguagem reconhece os operadores aritméticos de soma (`+`), subtração (`-`), multiplicação (`*`) e divisão (`/`), além do operador de atribuição (`=`).
- **Pontuação e Delimitadores:** São reconhecidos os símbolos de vírgula (`,`), ponto (`.`), ponto e vírgula (`;`), dois pontos (`:`), além de chaves (`{`, `}`) e parênteses (`(`, `)`) para aninhamento de instruções.
- **Comentários:** Delimitados estritamente por colchetes (`[` e `]`). O analisador reconhece e ignora formalmente qualquer conteúdo interno aos mesmos, sem gerar tokens.
- **Tratamento de Erros e Espaços:** Espaços em branco, quebras de linha e tabulações são ignorados, enquanto quebras de linha incrementam o contador de rastreamento. Qualquer caractere não previsto pelas regras descritas acima é capturado e gera o aviso de *Erro léxico*, indicando o caractere inválido e a linha de ocorrência.

2 Manual do Usuário

Para facilitar a compilação e a validação do analisador léxico, o projeto foi estruturado com scripts Bash de automação. É necessário possuir o compilador *GCC* e a ferramenta *Flex* devidamente instalados no sistema.

Compilação Rápida

Para gerar o analisador léxico a partir do código fonte, basta utilizar o script de compilação fornecido no diretório:

```
$ chmod +x compile.sh
$ ./compile.sh
```

O script `compile.sh` encarrega-se automaticamente de invocar o Flex sobre o arquivo `lexer.l` e, na sequência, utilizar o GCC para converter o código intermediário `lex.yy.c` no executável final do analisador.

Execução e Rotina de Testes

O analisador pode ser utilizado isoladamente apontando para qualquer arquivo de texto, sendo o arquivo `input1.txt` o explicitamente solicitado no enunciado do trabalho, que não contém erros léxicos. A execução é realizada da seguinte forma:

```
$ ./lexer input1.txt
```

Para validar todos os limites e tratamentos de erros discutidos durante o desenvolvimento, criamos um ambiente de teste em lote. Basta utilizar o script `run_tests.sh`:

```
$ chmod +x run_tests.sh
$ ./run_tests.sh
```

Este script percorrerá automaticamente os casos de teste rigorosos (de `input1.txt` até `input5.txt`), imprimindo as respectivas quebras de linha e saídas no terminal.

Para validar a integridade da implementação, consulte o arquivo auxiliar `expected_output.txt`. Ele contém o mapeamento e o gabarito detalhado de todos os tokens e erros léxicos que devem ser apresentados no terminal durante a execução da bateria de testes.

Repositório *Github*

Caso seja de interesse, todo o código fonte e arquivos para a realização deste trabalho estão disponível em https://github.com/gabrielmbrum/graduacao/tree/main/8th-semester/compiladores/trabalho1/.